PRÁTICAS EM SISTEMAS OPERACIONAIS

AULA 4

Prof. André Luiz de Souza Paula

CONVERSA INICIAL

Objetivo: conhecer comandos GNU/Linux que permitam que usuários e administradores localizem, acessem e gerenciem arquivos e diretórios de maneira eficiente.

Temas de estudo:

1. Gerenciamento de arquivos;
2. Comandos de manipulação de arquivos;
3. Comandos de busca;
4. Gerenciamento de permissões;
5. Comando de permissões.

TEMA 1 – GERENCIAMENTO DE ARQUIVOS

Os arquivos precisam ser mantidos tanto pelo sistema operacional, que garante a segurança e a integridade, quanto pelos seus usuários (manuseio). Portanto, vamos analisar os principais sistemas de arquivos, ferramentas e comandos que possibilitam a gestão compartilhada dos dados armazenados.

O objetivo desta seção é uma visão mais prática, do ponto de vista do usuário final, que precisa manipular seus arquivos. Portanto iremos abstrair algumas questões de gerenciamento do sistema operacional sobre o sistema de arquivos, e vamos apresentar as ferramentas de manipulação de arquivos e diretórios.

1.1 Sistema de arquivos (*Filesystem*)

O gerenciamento de arquivos inclui tanto o controle de acesso, controle da localização e manuseio dos conteúdos de cada arquivo. As funções de um sistema de arquivos se concentram no controle dos arquivos em disco. Dentro desse contexto as principais funções por ele desempenhadas são:

a. **Proteção:** manter a integridade dos dados armazenados;
b. **Mapeamento de nomes:** os arquivos são armazenados com identificador único e numérico dentro do disco, normalmente organizados em forma de árvore, hierarquicamente distribuídos;
c. **Endereçamento:** como os arquivos são alocados (e localizados) ao longo do disco;

d. Segurança: controle de permissão de acesso;

e. Contabilidade: arquivos de logs sobre eventos ocorridos (*Journaling*).

Entre os sistemas de arquivos suportados pelo GNU/Linux estão os sistemas a família ext{2,3,4}, xfs, jfs, reiserfs e btrfs, que apresentam excelentes tempos de resposta e possuem recursos avançados de recuperação de dados em casos de desastres e falha.

Os atributos dos arquivos variam de um sistema para outro, mas no ambiente GNU/Linux os principais são:

a. **Nome:** o nome simbólico do arquivo é a única informação conservada em forma legível pelas pessoas;

b. **Identificador:** este rótulo único, usualmente um número, identifica o arquivo dentro do sistema de arquivo;

c. **Tipo:** esta informação é necessária para os sistemas que suportam diferentes tipos de arquivos;

d. **Posição:** esta informação é um ponteiro para um dispositivo e para a posição do arquivo naquele dispositivo (*inode*);

e. **Tamanho:** o tamanho corrente do arquivo;

f. **Permissão:** a informação de controle de acesso determina quem pode ler o arquivo, gravá-lo, executá-lo e assim por diante;

g. **Hora e data:** estas informações podem ser conservadas em relação a data da criação, última modificação e última utilização do arquivo. Estes dados podem ser úteis para proteção, segurança e monitoramento de uso do arquivo;

h. **Identificação do usuário:** usuário dono, que criou o arquivo ou que teve o arquivo atribuído;

i. **Identificação do grupo:** grupo principal do usuário que criou o arquivo, ou grupo que teve o arquivo atribuído.

1.2 Navegação de arquivos sobre interface gráfica

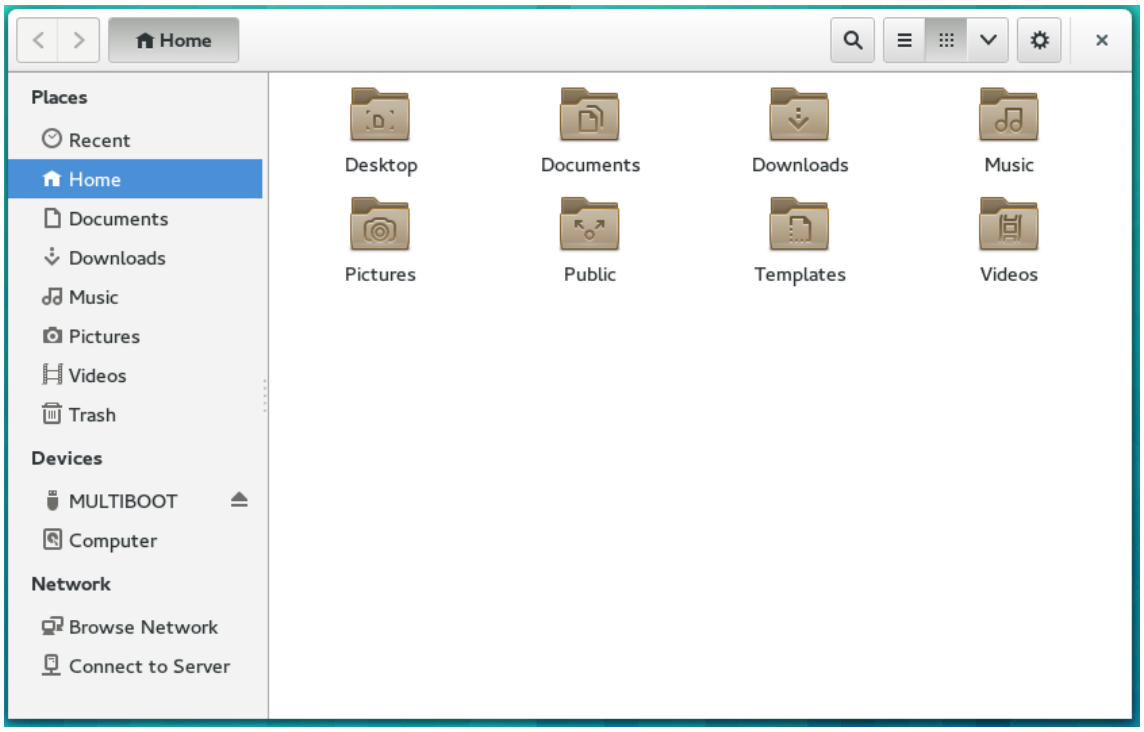
A interface gráfica é conhecida como GUI (*Graphical User Interface*) e oferece várias ferramentas de manipulação de arquivos e diretórios. Fazendo uma rápida relação com o MS-Windows, essas ferramentas seriam equivalentes ao MS-Explorer. Segue abaixo uma relação de ferramentas gráficas que permitem a navegação nos sistemas de arquivos GNU/Linux:

1.2.1 Gnome Files

Gnome Files, anteriormente e internamente chamado Nautilus, é o gerenciador de arquivos oficial do ambiente de desktop GNOME, funcionando principalmente para sistemas operacionais GNU/Linux. O *Gnome Files* é licenciado sob os termos da GNU (*General Public License*) e é software livre.

A área de trabalho gnome é uma área de trabalho padrão comum para a maioria das distribuições Linux e, como acontece com a maioria dos sistemas operacionais, você gerencia seus dados no gnome com um software chamado gerenciador de arquivos.

Em função da sua facilidade de uso e recursos adicionais, é largamente utilizado pela maioria das distribuições, a figura abaixo demostra sua aparência muito intuitiva para quem já utiliza o MS-Explorer:



Fonte: Paula, 2023.

1.2.2 Dolphin (KDE)

O Dolphin é o gerenciador de arquivos da KDE, outra opção de interface gráfica dos sistemas GNU/Linux, permite pelo conteúdo dos discos, dispositivos USB, cartões SD, dentre outros. Criar, mover ou deletar arquivos e pastas.

Possui funções de abas múltiplas e visualização dividida, que permite navegar por pastas diferentes ao mesmo tempo, facilitando para mover ou copiar.

O Dolphin é considerado um navegador de arquivos leve e tem suporte a três modos de visualização diferentes: visualização tradicional em grade de todos os arquivos, uma visualização mais detalhada, e visualização em árvore. O Dolphin ainda pode exibir arquivos e pastas de sistemas remotos.

1.2.3 Konqueror (KDE)

Conforme a descrição no Projeto KDE (*https://apps.kde.org/pt-br/konqueror/*), o Konqueror é o navegador Web do KDE e um canivete-suíço para gerenciamento e visualização de arquivos.

Funcionalidades:

a. Navegação Web usando KHTML ou KDEWebKit como mecanismo de renderização;
b. Gerenciamento de arquivos usando muitas das funcionalidades do Dolphin (incluindo controle de versão, menus de serviço e interface básica);
c. Gerenciamento de arquivos em servidores ftp e sftp;
d. Cliente FTP completo e repleto de recursos (você pode dividir a área de exibição para mostrar as pastas locais e remotas na mesma janela);
e. Visualização de arquivos com aplicativos incorporados (p.ex., Okular e Calligra para documentos, Gwenview para imagens, KTextEditor para arquivos de texto);
f. Vários tipos de *plugins*: Menus de serviço, KParts (aplicativos incorporados), KIO (acesso a arquivos usando protocolos especiais, como o http ou ftp) e plugins KParts (como o AdBlocker...).

1.2.4 Krusader (KDE)

Também mantido pelo projeto KDE, o Krusader é um navegador de arquivos mais avançado que o Konqueror. Segue abaixo a lista de funcionalidades apresentados pela KDE:

a. Manipulação de pacotes extensiva;
b. Suporte para sistemas de arquivos montados;
c. Suporte a FTP;

d. Módulo de pesquisa avançado;
e. Editor/visualizador interno;
f. Sincronização de pastas;
g. Comparação do conteúdo de arquivos;
h. Poderosa ferramenta para renomeação em lote;
i. Suporte para uma ampla variedade de formatos de arquivamento e manipulação de outros KIO *slaves*, como smb ou fish;
j. Personalizável e intuitivo.

1.2.5 Outros gerenciadores de arquivos

Thunar, usado no ambiente desktop LXDE, e o Xfe. Ambos prometem entregar um navegador de arquivos simples, rápido e com funcionalidades básicas para ambientes mais enxutos e com poucos recursos.

1.3 Tipos de arquivos

No GNU/Linux, um arquivo pode ser comum ou especial. Estes são os principais tipos de arquivos que podemos encontrar:

a. **arquivo comum** – armazena informações e pode ser de texto, de áudio, de imagem, binários, entre outros formatos;
b. **diretório** – contém outros arquivos e/ou outros diretórios;
c. **dispositivo** – representa um dispositivo físico (hardware) do sistema. Existem dois tipos:

- **de caracteres** – as operações de E/S são feitas de forma sequencial.
- **de blocos** – as operações de E/S são feitas usando blocos de caracteres.

d. **link simbólico** – aponta para outro arquivo.
e. **pipe** – utilizado para comunicação entre processos.
f. **socket** – utilizado para comunicação entre processos.

A tabela abaixo mostra como os tipos de arquivos são referenciados dentro do ambiente GNU/Linux:

Tipo	Representação
Arquivo comum	-
Diretório	d
Dispositivo de caracteres	c
Dispositivo de blocos	b
Link simbólico	l
Pipe	p
Socket	s

A extensão de um arquivo sempre foi um fator relevante para identificar o tipo de arquivo, do ponto de vista do usuário. A interface gráfica também utiliza a extensão do arquivo para associar com um programa e realizar seu tratamento (acesso ou alteração). Contudo, nem sempre a extensão do arquivo revela a sua versão, e em alguns casos o arquivo não possui extensão. Para sanar este problema vamos usar formas diferentes de identificar o tipo do arquivo e a sua associação:

1. Identificar o arquivo através da sua extensão:

```
~# ls -l
-rw-r--r-- 1 root root    3028 Nov 30 2022 add.conf
drwxr-xr-x 4 root root    4096 Jan 21 2023 script.sh
drwxr-xr-x 1 root root    4096 Jan 21 2023 texto.txt
```

2. Identificar o tipo de arquivo com o comando file:

```
~$ file --mime-type -b fonte.txt
fonte.txt: text/plain
```

3. Identificar o arquivo com o comando stat:

```
  ~# stat main.bash
  File: main.bash
  Size: 203             Blocks: 8          IO Block: 4096
regular file
Device: 801h/2049d      Inode: 277379      Links: 1
Access: (0777/-rwxrwxrwx Uid: (0/root)   Gid: (0/root)
Access: 2023-11-24 04:09:03.299588804 +0000
Modify: 2023-11-24 04:09:03.291588731 +0000
Change: 2023-11-24 04:09:03.295588767 +0000
Birth: 2023-11-24 04:09:03.291588731 +0000
```

4. Identificar qual o programa está associado ao tipo do arquivo:

Os sistemas GNU/Linux utilizam o comando *update-alternatives* para administrar os programas padrão que irão manipular os arquivos. Para exemplificar vamos definir qual será o programa padrão para manipular arquivos de texto:

4.1 Identificando quais programas de edição de texto puro (*text/plain*) estão disponíveis:

```
~# update-alternatives --list editor
/bin/nano
/usr/bin/vim.basic
/usr/bin/vim.tiny
```

4.2 Configurando o programa padrão que será o “editor” de arquivos de texto (*text/plain*):

```
~# update-alternatives --config editor
There are 3 choices for the alternative editor:
Selection      Path                Priority   Status
------------------------------------------------------------
*0            /bin/nano            40        auto mode
 1            /bin/nano            40        manual mode
 2            /usr/bin/vim.basic   30        manual mode
 3            /usr/bin/vim.tiny    15        manual mode
Press <enter> to keep the current choice[*], or type selection
number:
```

TEMA 2 – COMANDOS DE MANIPULAÇÃO DE ARQUIVOS

Nesta seção iremos detalhar os principais comandos de manipulação de arquivos e diretórios, com objetivo de facilitar a administração do sistema de arquivos. Este é o conteúdo básico para acessar os arquivos e diretórios, listar o seu conteúdo, alterar e remover, utilizando um terminal de linha de comando (*SHELL)*.

Todos os comandos que serão apresentados oferecem um manual e um *help online*:

~# man 1 <comando>

~# <comando> --help

2.1 Comando *cd* (*Change Directory*)

O comando cd troca o atual diretório de trabalho, com este comando é possível navegar (entrar e sair) nos diretórios do sistema de arquivos por linha de comando.

Manual: *https://man7.org/linux/man-pages/man1/cd.1p.html*

Principais opções:

~	Retorna ao diretório HOME do usuário
-	Retorna ao último diretório acessado
..	Volta ao diretório pai (anterior)
/	Volta ao diretório raiz (/)

Sintaxe:

~# cd <destino> Change Directory

Exemplo:

~# cd /home/aluno

/home/aluno#

O comando acima entra no diretório /home/aluno, passa a ser o “diretório de trabalho”.

2.2 Comando *mkdir* (*Make Directory*)

O comando “mkdir” cria diretórios, também pode criar uma sequência de diretórios (com a opção “-p”).

Manual: *https://man7.org/linux/man-pages/man1/mkdir.1.html*

Principais opções:

-m	Determina as permissões (modo octal)
-p	Cria todo caminho informado (*parentes*)
-v	Mostra cada diretório que criar (*debug*)

Sintaxe:

~# mkdir <caminho>\<nome>

Exemplo:

~# mkdir /home/aluno/aula

O comando acima cria o diretório /home/aluno/aula.

2.3 Comando *rmdir* (*Remove Directory*)

O comando "*rmdir*" remove diretórios **vazios** (apenas).

Manual: *https://man7.org/linux/man-pages/man2/rmdir.2.html*

Principais opções:

-p	Remove uma cadeia de diretórios (*parentes*)
-v	Mostra cada diretório que remover (*debug*)

Sintaxe:

~# rmdir <caminho>\<nome>

Exemplo:

~# rmdir /home/aluno/aula

O comando acima remove o diretório aula (se estiver vazio).

2.4 Comando *ls* (*list*)

O comando "ls" lista o conteúdo dos diretórios.

Manual: *https://man7.org/linux/man-pages/man1/ls.1.html*

Principais opções:

-a	Mostra arquivos ocultos
-c	Ordena os arquivos por ordem decrescente de criação
-d	Lista apenas diretórios
-l	Retorna informações detalhadas em forma de colunas
-h	Retorna o tamanho dos arquivos em KB, MB, GB e TB
-S	Lista os arquivos por tamanho (maiores primeiro)

Sintaxe:

~# ls <caminho>

Exemplo:

~# ls /home/aluno/aula

O comando acima lista o conteúdo do diretório /home/aluno/aula.

2.5 Comando *cp* (*Copy*)

O comando “cp” copia arquivos e diretórios de um local para outro.

Manual: *https://man7.org/linux/man-pages/man1/cp.1.html*

Principais opções:

-a	Preserva todas as propriedades do arquivo de origem
-f	Copia mesmo que os arquivos já existam no destino
-p	Preserva apenas as mesmas permissões da origem
-r	Recursivo, copia todos os arquivos e subdiretórios
-v	Mostra o passo a passo da execução do comando (*debug*)

Sintaxe:

~# cp <origem> <destino>

Exemplo:

~# cp /etc/hosts /home/aluno/

O comando acima copia o arquivo hosts para dentro do diretório /home/aluno.

2.6 Comando *mv* (*Move*)

O comando “mv” movimenta o arquivo/diretório (removendo da origem) para um novo local. Também serve para renome para arquivos ou diretórios.

Manual: *https://www.man7.org/linux/man-pages/man1/mv.1.html*

Principais opções:

-f	Força a execução se houver alguma restrição
-u	Atualiza o arquivo no destino, se ele já existir
-v	Mostra a execução de forma detalhada (*debug*)

Sintaxe:

~# mv <origem> <destino>

Exemplo:

~# mv /home/aluno/notas.txt /home/aluno/pso/aula/

O comando acima move o arquivo hosts para dentro do diretório /home/aluno/pso/aula/.

2.7 Comando *rm* (Remove)

O comando “rm” remove arquivos e diretórios. Use este comando apenas se tiver certeza de que os arquivos não são mais necessários, pois não poderá recuperar novamente.

Manual: *https://man7.org/linux/man-pages/man1/rm.1.html*

Principais opções:

-f	Força a execução, mesmo que haja alguma restrição
-i	Solicita confirmação antes de remover o arquivo
-r	Remove de forma recursiva, removendo subdiretórios
-v	Mostra a execução de forma detalhada (*debug*)

Sintaxe:

~# rm <caminho>\<nome>

Exemplo:

~# rm /home/aluno/texto.txt

O comando acima remove o arquivo texto.txt.

TEMA 3 – COMANDOS DE BUSCA

Os comandos de busca têm a finalidade de varrer o sistema de arquivos para encontrar um arquivo ou diretório. A pesquisa pode ser feita pelo nome do arquivo, tamanho, dono, permissões, entre vários outros parâmetros.

3.1 Comando locate

O comando *locate* utiliza um banco de dados (*/var/cache/locate/locatedb*) que precisa ser atualizado antes da pesquisa, e precisa ter um pacote de *software* instalado chamado *locate*.

Para atualizar seu banco de dados para pesquisa é necessário rodar o comando: ***~# updatedb***

Após a sua atualização é possível fazer buscas com o comando locate: ***~# locate <nome do arquivo/diretório a ser localizado>***

O comando Linux locate pode retornar até arquivos que foram deletados. Isso vai ocorrer se o seu banco de dados estiver desatualizado, e será necessário executar o comando *updatedb* novamente para atualizar.

Existem algumas opções como “-c” que retorna o número de ocorrências encontradas, em vez do nome da localização do arquivo/diretório. E “-b” para evitar que retorne diretórios, mostrando apenas arquivos.

Exemplo de uso:

~# updatedb

~# locate hostname

/etc/hostname

3.2 Comando find

O comando FIND é o melhor comando para buscar informações no GNU/Linux. Vamos usar vários exemplos de como utilizar este comando, mas pode começar lendo o seu manual: *~$ man find*

find ***<onde?> <como?> <o quê?>***

<onde?> informe em qual diretório deseja procurar

<como?> informe qual critério deseja usar (nome, tamanho, permissão...)

<o quê?> informe um parâmetro de busca

Exemplos:

1. *~$ find /home -name Projeto.docx*

Este comando fará uma busca pelo arquivo “Projeto.docx” no diretório /home.

2. ~$ *find /home -iname Projeto.docx*

O iname não é case sensitive, e busca com caixa alta ou baixa.

3. *~$ find /home –size +100M*

Este comando fará uma busca por arquivos maiores que 100MBytes.

4. *~$ find /home –size -100K*

Este comando fará uma busca por arquivos menores que 100KBytes.

5. ~$ *find /home –ctime 2*

Este comando fará uma busca por arquivos criados nas últimas 48 horas.

6. ~$ *find /home –executable*

Este comando fará uma busca por arquivos executáveis.

7. ~$ *find /home –type f –exec du –sh {} *

Este comando irá executar o subcomando “du -sh” em todos os arquivos do /home e mostrará seu tamanho.

8. ~$ *find /home -type f -exec grep -l seutexto {} *

Este comando vai procurar uma string (“seutexto”) nos arquivos do /home.

TEMA 4 – GERENCIAMENTO DE PERMISSÕES

Os sistemas operacionais GNU/Linux são sistemas multiusuários, e para garantir que os usuários tenham acesso somente aos recursos para os quais eles podem utilizar configuramos as permissões de acesso a arquivos, diretório e outros dispositivos. Vários fatores podem alterar a forma como configuramos o sistema de permissões como tipo de permissão, tipo de usuário, grupo, tipo de arquivo, entre outros elementos.

O primeiro exemplo dessa preocupação pode ser observado no Sistema de Arquivos (*FHS*), onde cada usuário tem seu próprio diretório: *$ ls -l /home*

4.1 Principais atores do sistema de permissões: Dono, Grupo e Outros

Um arquivo, ou diretório, possui informações de controle que são chamados de metadados, entre essas informações vamos obter a quem esse arquivo pertence e quais as permissões que foram cedidas pelo administrador do sistema:

Exemplo 1

```
~$ stat /home/aluno
  File: /home/aluno
  Size: 12288               Blocks: 24            IO Block: 4096
directory
Device: 801h/2049d      Inode: 662342      Links: 29
Access: (0755/drwxr-xr-x) Uid: (1000/ aluno) Gid: (2001/ pso)
Access: 2023-11-22 15:34:36.906874260 -0300
Modify: 2023-11-08 10:18:25.728705913 -0300
Change: 2023-11-08 10:18:25.728705913 -0300
Birth: -
```

Exemplo 2

```
~$ ls -l /home/suporte/.profile
-rw-r--r—1 aluno pso 807 Aug  7  2020 /home/suporte/.profile
```

Usuário dono (*UID*)

O usuário dono é identificado pelo UID (*User IDentification*), significa que este usuário terá permissões específicas de acesso a este arquivo, por padrão com permissão de leitura e escrita. Este usuário também poderá alterar as permissões do arquivo, uma vez que detém a sua “propriedade”.

Nos exemplos 1 e 2, acima, o dono dos arquivos é o usuário “aluno” (*UID* 1000).

Grupo do arquivo/diretório (*GID*)

O grupo do arquivo/diretório é identificado pelo GID (*Group IDentification*), significa que este grupo terá permissões específicas de acesso a este arquivo, por padrão com permissão de leitura. As permissões de grupo serão válidas apenas para os usuários que fazem parte do grupo, exceto o usuário dono que possui permissões específicas.

Nos exemplos 1 e 2, acima, o dono dos arquivos é o grupo “pso” (*GID* 2001).

Outros

Os “outros” são usuários que não tem nenhum vínculo com o arquivo, não é dono (*uid*) e não faz parte do grupo (*gid*). Nos exemplos 1 e 2, acima, qualquer usuário que não seja o “aluno” e não faça parte do grupo “pso”. Existem permissões específicas para esses usuários.

Resumindo, existem permissões específicas para cada ator (dono, grupo e outros). Nas próximas seções serão identificados quais são as permissões que podem ser aplicadas a cada um desses atores do sistema de permissão.

4.2 Tipos de permissões (modo octal)

As permissões **básicas** são:

1. **Leitura (*read*):** permite que os dados armazenados dentro do arquivo sejam lidos. Por exemplo, permite que um arquivo de texto possa ser lido pelo comando “cat”: *~$ cat /etc/hosts*;

2. **Escrita (Write):** permite alteração nos dados armazenado no arquivo. Entretanto, a permissão de escrita em um diretório significa dizer que eu posso criar ou remover novos arquivos dentro deste diretório;
3. **Execução(eXecution):** permite que arquivos binários e scripts possam ser executados, voltado para programas executáveis e scripts. A execução é basicamente criar um processo dentro do sistema operacional para uma que uma determinada tarefa (programa neste arquivo) possa ser executada.

O **modo octal** tem relação com o sistema de numeração octal (base 8), onde encontramos os dígitos 0, 1, 2, 3, 4, 5, 6 e 7. Cada tipo de permissão recebe um valor, conforme a tabela a seguir:

r – **Read** (leitura)	w – **Write** (escrita)	x – **eXecution** (execução)
4	2	1

Observe que vamos identificar **Leitura** com a letra "**r**", **Escrita** com "**w**" e **Execução** com "x". Na tabela abaixo vamos somar os valores das permissões atribuídas. Por exemplo:

0 (zero) representa que nenhuma permissão foi dada (`---`);

1 (um) representa que somente a permissão de execução foi atribuída (`--x`)

Permissões	**Octal**
- - -	0
- - x	1
- w -	2
- w x	3
r - -	4
r – x	5
r w -	6
r w x	7

Observe o comando que será analisado nos exemplos 3, 4 e 5:

```
aluno@computador:~/curso$ ls -l
total 29604
-rw-r--r-- 1 mario   pso   2001 Jan  5    10:58 Curso.pdf
drwxrwxrwx 5 joao    prof  44   Jan  12   14:46 DebianDesktop
```

```
-rwxr-xr-x 1 maria   pso   5598 Jan  12   15:24 Desktop.sh
-rw------- 1 maria   pso   1311 Jan  4    09:12 Expresso.pdf
drwxrwxrwx 3 jose    pso   19   Jan  12   14:46 Pessoa
-rw-r--r-- 1 aluno   pso   6810 Jan  13   10:50 SL2.odp
drwxrwxrwx 3 root    root  17   Jan  12   14:46 Totais
```

Exemplo 3

O arquivo **Curso.pdf** tem as seguintes permissões: `rw- r-- r--`

1. Dono: r w – (Leitura 4 + Escrita 2 = Modo octal 6)
2. Grupo: r w – (Leitura 4 = Modo octal 4)
3. Outros: r w – (Leitura 4 = Modo octal 4)

Exemplo 4

O arquivo **Desktop.sh** tem as seguintes permissões: `rwx r-x r-x`

4. Dono: r w x (Leitura 4 + Escrita 2 + Execução 1 = Modo octal 7)
5. Grupo: r - x (Leitura 4 + Execução 1 = Modo octal 5)
6. Outros: r – x Leitura 4 + Execução 1 = Modo octal 5)

Exemplo 5

Permissão	Tipo de arquivo	User (u)	Group (g)	Other (O)	Octal
-rw-r--r--	-	rw- (6)	r--(4)	r--(4)	644
-rwxr-xr-x	-	rwx (7)	r-x(5)	r-x(5)	755
-rw-r--r--	-	rw-(6)	r--(4)	r--(4)	644
crw-------	C	rw-(6)	---(0)	---(0)	600
lrwxrwxrwx	L	rwx(7)	rwx(7)	rwx(7)	777
brw-rw-r--	B	rw-(6)	rw-(6)	r--(4)	664

TEMA 5 – COMANDOS DE PERMISSÕES

5.1 Comando chmod (*Change Mode*)

Muda as permissões de um arquivo ou diretório.

Sintaxe: *chmod <permissões> <arquivo/diretório>*

Exemplos:

```
~# chmod 755 script.sh      (Altera as permissões do arquivo script.sh)
~# chmod u+rwx script.sh    (Atribui as permissões r w x para o dono → u)
~# chmod g+rx script.sh     (Atribui as permissões r x para o grupo → u)
~# chmod o+r script.sh      (Atribui somente r para outros → u)
~# chmod o-rwx script.sh    (Remove todas as permissões para outros)
```

5.2 Comando chown (*Change Owner*)

O comando chown permite que se altere o usuário dono de um arquivo, ou diretório. Sintaxe: *chown <usuário> <arquivo/diretório>*

Exemplo: *~# chown aluno script.sh*

5.3 Comando chgrp (*Change Group*)

Muda o grupo ao qual um arquivo ou diretório faz parte.

Sintaxe: *chgrp <grupo> <arquivo/diretório>*

Exemplo: chgrp pso script.sh

5.4 Comando umask

Define uma máscara de permissão pela qual os novos arquivos e diretórios serão criados.

Sintaxe: *umask* <valor> (valor padrão 022)

A tabela a seguir exemplifica como calcular a permissão padrão usando a máscara (*umask*) configurada:

	Diretório	**Arquivo**
Permissão inicial fixo	777	666
UMASK (022)	022	022
Valor padrão	**755**	**644**

5.5 Permissões especiais

5.5.1 SETUID

Se este bit estiver ligado em um arquivo executável, isso indica que que ele vai rodar com as permissões do seu dono (o proprietário do arquivo) e não

com as permissões do usuário que o executou (Só tem efeito em arquivos executáveis.)

Como aplicar: *~# chmod **4**755 <arquivo executável>*

5.5.2 SETGUID

Faz o mesmo que o Setuid, mas agora o arquivo executado vai rodar com as permissões do grupo do arquivo. Se aplicado a um diretório, força os arquivos/diretórios criados dentro dele a ter o mesmo grupo do diretório pai, ao invés do grupo primário do usuário que o criou.

Como aplicar: *~# chmod **2**755 <diretório>*

5.5.3 STICKY

Se este bit for ativado, faz com que arquivos criados em um determinado diretório só possam ser apagados por quem o criou (dono) ou pelo super-usuário do sistema. Um exemplo de uso do Sticky é o diretório /tmp.

Como aplicar: *~# chmod **1**755 <arquivo / diretório>*

FINALIZANDO

Com objetivo de fixar conhecimento sobre os principais termos utilizados nesta etapa, vamos listar abaixo os principais conceitos revisados:

- **Sistema de arquivos *(Filesystem)*:** um sistema de arquivos é uma estrutura usada por um sistema operacional para organizar e gerenciar arquivos em um dispositivo de armazenamento, como um disco rígido, unidade de estado sólido (SSD) ou pendrive. Ele define como os dados são armazenados, acessados e organizados no dispositivo de armazenamento;
- ***Graphical User Interface (GUI)*:** é uma interface digital na qual o usuário interage com componentes gráficos como ícones, botões e menus;
- ***Command Line Interface (CLI)*:** é uma interface baseada em texto onde você pode inserir comandos que interagem com o sistema operacional de um computador;

- **Gerenciador de arquivos**: o gerenciador de arquivos é uma ferramenta do sistema operacional que facilita manter o controle dos arquivos e pastas;
- **Dispositivo de caracteres:** lê ou escreve uma sequência de caracteres. Cada caractere não possui endereço e não pode ser acessado aleatoriamente. Para cada caractere (ou conjunto pequeno deles), é gerada uma interrupção. Exemplos: Teclado, Mouse, entre outros;
- **Dispositivo de blocos:** transferem ou armazenam dados em blocos de tamanho fixo, cada bloco possui um endereço e cada bloco pode ser acessado independentemente (acesso aleatório). Exemplos: Discos Rígidos (SCSI, IDE, SATA e SAS), CD/DVD, Fitas, SSDs, entre outros.
- **Páginas de manual (man):** uma interface para os manuais de referência on-line. Permite que o usuário consulte o manual de um comando GNU/Linux:
- *~# man <comando>*
- **Help online:** uma opção que os comandos oferecem que permite que o usuário consulte um guia rápido (manual resumido) das principais opções:
- *~# <comando> --help*
- **Permissão de arquivos:** Uma permissão é um atributo de um arquivo, diretório, processo ou dispositivo, que gerencia o acesso, alteração e até a remoção deste objeto. A permissão é atribuída a um usuário ou grupo;
- ***User IDentification (UID)*:** Nos sistemas operacionais GNU/Linux, *UID* significa Identificador de Usuário. É um valor numérico exclusivo atribuído a cada conta de usuário;
- ***Group IDentification(GID)*:** Nos sistemas operacionais GNU/Linux, *GID* significa Identificador de Grupo. É um valor numérico exclusivo atribuído a cada grupo do sistema;
- ***Modo Octal*:** O modo octal é um conjunto de oito números onde cada número define um tipo de acesso diferente para usuário dono, grupo e outros de um arquivo/diretório;
- ***SETUID BIT (set-user-ID)***: Usado para ativar uma permissão especial em um arquivo executável, qualquer usuário que executar este arquivo executará com as mesmas permissões do usuário dono;
- ***SETGUID BIT (set-group-ID)***: Usado para ativar uma permissão especial, onde um diretório compartilhado mantém permissões de acesso e escrita

para o seu grupo, independente de qual usuário criar um objeto dentro deste diretório;

- ***STICKY BIT*:** Usado para proteger os arquivos criados por um usuário em um diretório compartilhado. Esta opção não permite que um usuário delete arquivos de outros usuários.

REFERÊNCIAS

DEITEL, P. J., DEITEL, H. M., CHOFFNES, D. R. **Sistemas operacionais**. 3. ed. São Paulo: Pearson, 2017.

HILL, B. M.; BACON, J. **O Livro Oficial do Ubuntu**. 2. ed. Porto Alegre: Bookman, 2008.

LINUX.ORG. Disponível em: <https://www.linux.org/>. Acesso em: 22 fev. 2023.

MACHADO, F. B.; MAIA, P. L. **Fundamentos de sstemas operacionais**. Rio de Janeiro: LTC, 2011.

NEMETH, E.; SNYDER G.; HEIN T. R. **Manual completo do Linux**: guia do administrador. 2. ed. São Paulo: Pearson Prentice Hall, 2007.

TANENBAUM, A. S. **Sistemas operacionais modernos**. 4. ed. São Paulo: Pearson Prentice Hall, 2008.

SILBERSCHATZ, A.; GALVIN, P. B.; GAGNE, G. **Fundamentos de sistemas operacionais**. 9. ed. Rio de Janeiro: LTC, 2015.

SILVA, R. L. **Linux**: guia de instalação e administração. São Paulo: Novatec, 2010